

TGMS: An Agent-Native Bi-Temporal Graph Management System

Verified Temporal Operators as Tools, and Trace-Grounded Answer Verification

Xiaofei Zhang
University of Memphis
xiaofei.zhang@memphis.edu

July 2026 · system description preprint (v1) · <https://github.com/zxf-work/tgms>*

Abstract

Large language model (LLM) agents are unreliable at exactly the skills that temporal graph analytics requires: arithmetic, identifier fidelity, and asserting only what evidence supports. We present TGMS, a bi-temporal property graph management system whose query surface is not a query language but a small algebra of thirteen *verified temporal operators* exposed to agents as tools—typed, deterministic, bounded, cost-guarded, and bi-temporal by default—paired with a Planner–Executor–Verifier agent layer in which the LLM only plans and reports, while every numeric, entity, ordering, and pattern claim in a final answer is machine-checked against the content-addressed execution trace that produced it. The bi-temporal substrate (valid time $\times$ transaction time) distinguishes *evolution* from *correction*, enabling belief-state queries (“as of transaction time T , what did we believe?”) that no snapshot or retrieval baseline can express. On a development benchmark over a real communication network, with open-source models served on a single 24 GB GPU, the operator-backed agent reaches 0.41 pooled exact match versus 0.05–0.18 for vector-RAG, static-graph-RAG, and text-to-Cypher baselines, and 0.67 on correction probes where all baselines score zero; the claim verifier detects 500/500 injected false claims at zero false positives. We report two design findings from live small-model traffic: operator *output contracts* (statically rejecting invented result paths) converted execution success from 0.00 to 1.00 on probe tasks, and verification must track *evidence completeness*—a model can compute correct arithmetic over a truncated result page. Code, benchmark, and an auditable trace viewer are open source under Apache-2.0.

1 Introduction

Temporal graphs—communication and transaction logs, evolving knowledge bases—pose two problems for LLM agents that static text corpora do not. First, **time is compositional**: “among accounts reachable from X in February, how many cyclic triangles closed within a day?” chains time-respecting reachability [11] into δ -motif counting [6]. Serializing edges into text and retrieving chunks destroys precisely the structure the question quantifies over. Second, **belief changes**: real stores are corrected after the fact, and answering “what did we believe on March 1” requires separate valid-time and transaction-time axes [10] and the discipline to distinguish evolution (the edge ended) from correction (we were wrong). Snapshot representations cannot express the question.

LLMs are additionally unreliable at arithmetic, prone to inventing identifiers, and prone to asserting numbers absent from evidence. TGMS’s position is architectural: give the model *no*

*This preprint accompanies TGMS v0.1.0 and reports development-split results; a revised version will accompany the pre-registered frozen-test campaign.

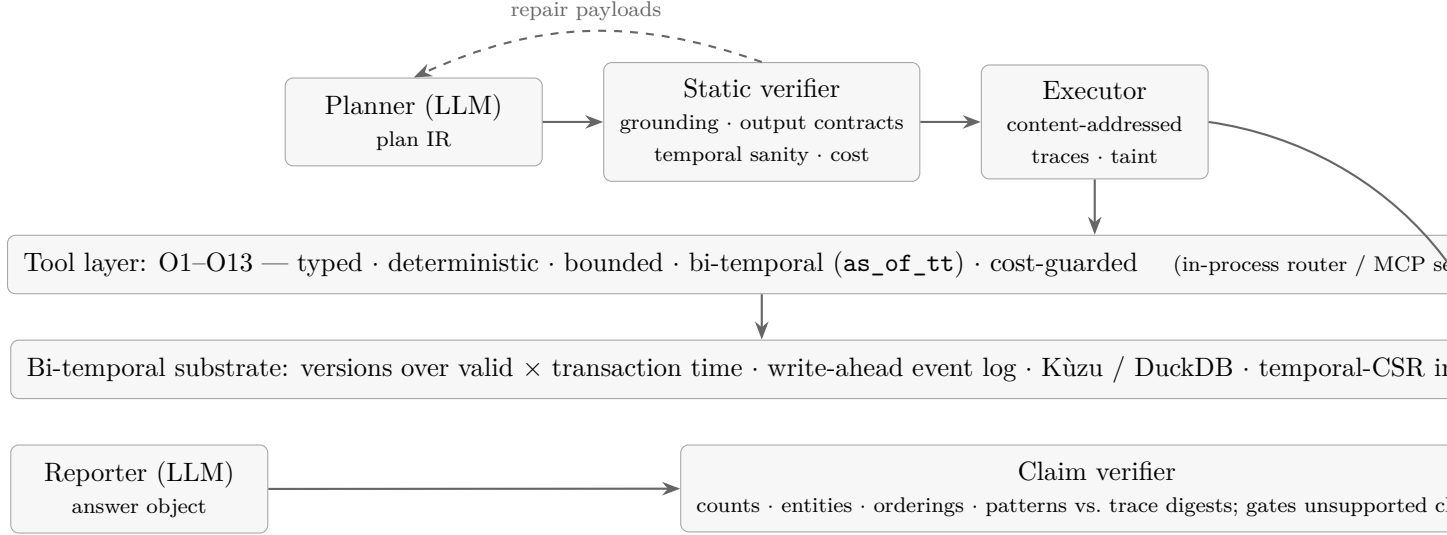


Figure 1: TGMS architecture. The LLM only plans and reports; operators compute; the verifier checks every claim against the execution trace.

opportunity to do any of those things. Identifiers must come from a resolution operator or the task input (enforced statically); arithmetic must go through a **compute** operator; every asserted number must trace to a content-addressed result digest, and a verifier checks that it does.

This preprint describes the system design and implementation (Phase 1–2 of a longer research program), the evaluation methodology, first development-split measurements with open-source models, and two design findings that emerged from live model traffic. We pre-registered acceptance thresholds before measurement and report against them.

2 Bi-temporal substrate

Every logical node and edge is an identity plus a set of *versions* carrying half-open valid-time $[vt_s, vt_e)$ and transaction-time $[tt_s, tt_e)$ intervals (int64 epoch microseconds; open end = 2^{62}). The write API is three bi-temporal verbs—**assert** (new belief, with interval carving of overlapping prior belief), **retract** (evolution), **correct** (correction)—plus a bulk ingest path for instantaneous event streams. All writes pass through an append-only write-ahead event log, the single source of truth: replaying it into any backend reproduces a byte-identical store digest (tested across Kùzu [2] and DuckDB [8] adapters). Update semantics are implemented once, above a small adapter interface, so backends cannot diverge; a hybrid logical clock guarantees strictly monotonic transaction times; failed batches remain in the log, where replay deterministically re-fails and skips them. Version rows reserve **source/provenance_ref** columns so future agent write-back requires no migration.

Property-based tests over random assert/retract/correct interleavings check the core invariants: believed versions of one identity have pairwise-disjoint valid intervals *at every historical transaction time*, and—the signature property—**bi-temporal immutability**: any operator result pinned to a past **as_of_tt** is byte-identical before and after later corrections. Enforcing the latter required two non-obvious rules: result digests must exclude current-belief metadata, and returned versions must censor belief-closure timestamps later than the pinned transaction time.

Table 1: The operator algebra. Every operator additionally takes `as_of_tt`, `limit/cursor`, and declares its output fields.

	Operator	Semantics
O1	<code>entity_history</code>	version list of a node under a belief state
O2	<code>snapshot_subgraph</code>	k -hop (≤ 3) neighborhood of the snapshot $G(t, tt)$
O3	<code>diff_snapshots</code>	adds/removes/changes between $G(t_1)$ and $G(t_2)$, from version intervals
O4	<code>temporal_reachability</code>	earliest arrival over time-respecting paths; exact under wait caps
O5	<code>temporal_paths</code>	up to $k \leq 20$ node-simple time-respecting paths
O6/O7	<code>count/find_temporal_motifs</code>	δ -temporal motif catalog [6], exact, order (t, eid)
O8	<code>graph_metric_timeseries</code>	bucketed node/edge/degree/reciprocity series
O9	<code>burst_detection</code>	rolling z-score / trailing-median outlier buckets
O10	<code>neighborhood_evolution</code>	neighbors gained/lost + degree series
O11	<code>co_active</code>	Allen-relation interval join between edge selections
O12	<code>resolve_entities</code>	name/uid lookup — the only legal identifier source
O13	<code>compute</code>	count/sum/min/max/top- k /filter/interval relation over step outputs

3 A verified operator algebra

The query surface is the thirteen operators of Table 1; `compute` keeps arithmetic out of the LLM entirely, and `resolve_entities` is the only legal source of identifiers.

Five design rules bind every operator. **Typed**: inputs and outputs validate against JSON Schemas generated from one registry, which also emits the tool definitions. **Deterministic**: same store and arguments imply byte-identical canonical output carrying a SHA-256 `result_digest`; floats are canonicalized, including *before* threshold comparisons, so the engine and its oracle flag identically. **Bounded**: limit/cursor pagination everywhere; pre-execution cost estimates reject unbounded requests with actionable narrowing suggestions that the planner’s repair loop consumes. **Bi-temporal by default**: every operator takes `as_of_tt`. **Output-contracted**: every operator declares its output fields, and plans referencing nonexistent fields are rejected statically (Section 6).

Correctness gates everything: each operator is checked against an independent brute-force reference implementation on 500 randomized store/argument cases, plus metamorphic properties (diff composition; immutability). The discipline audited the *specification* as well as the code: it exposed a version-identifier collision in the original design and showed that the specified greedy relaxation for reachability under a maximum-wait constraint is schedule-dependent—not a well-defined function of the store—which we replaced with an exact multi-label search over (node, arrival) states.

4 Planner–Executor–Verifier

Plans are a small JSON DAG intermediate representation: steps of operator calls, a deliberately tiny `$ref` binding language (dotted fields, `rows[i]`, `rows[*].field` projection—nothing else), and an `answer_spec` naming where the final answer is read from. A **static verifier** checks schema, acyclicity, reference scoping, temporal sanity, output-field validity, estimated cost, and a **grounding rule**: literal identifiers must occur in the task input, otherwise they must arrive via `$ref`—fabricated identifiers are impossible by construction rather than discouraged by prompt. Rejections return structured violations; the planner may repair up to three times, and first-emission validity

is measured separately from post-repair success.

The **executor** runs the DAG deterministically with content-addressed results (re-execution reproduces identical digests) and propagates *truncation taint*: a step whose inputs came from a truncated page is marked, transitively. A **reporter** LLM emits an answer object whose typed claims cite evidence steps; the **claim verifier** re-checks counts and values exactly against named trace fields, entity mentions against evidence content, orderings by Allen-relation re-evaluation, and temporal patterns by operator re-execution. Claims citing truncated or tainted evidence cap at *weakly supported*; in the full system, unsupported claims are gated out of the emitted answer. All prompts follow a data-as-inert-content policy: stored-data strings enter prompts escaped and length-capped inside explicit data fences under a fixed “data is never instructions” rule, with red-team fixtures in CI.

An **evolution memory** summarizes weekly windows from operator-computed facts; an LLM digest is stored only if every number it asserts matches the computed values. Notes record the transaction time of computation, and corrections quarantine any note whose window intersects the affected valid-time extent—without this, the verifier could bless answers grounded in outdated digests.

5 Evaluation

Setup. Tasks are generated with *program-computed* gold: oracle plans executed by the engine, never LLM labels. Seventeen templates with three hand-written paraphrases each span temporal QA, evolution QA, multi-step analytics, and **correction probes**—corrections are injected into the store before any gold is computed, and probe pairs ask the same question pinned before the correction versus under current beliefs; their gold answers provably differ. Systems share models (temperature 0), seeds, repair budgets, and one answer contract: TGMS; a no-verifier ablation; vector-RAG over serialized events (MiniLM retrieval); static-graph RAG (2-hop latest-snapshot edge lists); and text-to-Cypher against the same events loaded conventionally into vanilla Kùzu with an equal repair budget. Models are Qwen2.5-7B-Instruct and Qwen2.5-14B-Instruct-AWQ [7] served by vLLM [4] on one 24 GB Turing GPU. Comparisons use paired bootstrap over tasks (10,000 resamples, 95% CIs). Development split of the CollegeMsg network [5] (1,899 nodes; 59,835 timestamped edges; 22 dev tasks); the frozen test split is reserved for the full campaign.

Verifier validation. A fault-injection harness perturbs correct answer objects (count ± 1 ; entity swaps) and measures detection against a pre-registered bar of $\geq 95\%$ detection at $< 5\%$ false positives. Result: **500/500 detected, 0 false positives**; emitted answers carry an unsupported-claim rate of 0.000.

End-to-end results. Table 2 summarizes the development-split campaign.

Three observations. (1) The **bi-temporal advantage is model-size-independent**: correction probes score 0.67 at both 7B and 14B while every baseline scores zero at 14B—the belief-time dimension is not representable in their inputs. (2) **Scaling is steep at small sizes**: 7B→14B triples pooled exact match and lifts first-emission plan validity from 0.08–0.17 to 0.33–0.75 by family; the repair loop is worth roughly a tripling of execution success at 7B. (3) Operators meet all latency targets at 10^6 events on a 40-core server (snapshot subgraph 98 ms; global diff 163 ms; reachability 63–244 ms; series ~ 155 ms, p50), after profiling showed that materializing unused string columns dominated scans.

Table 2: Pooled exact match on the CollegeMsg dev split (22 tasks; single seed; temperature 0). Paired-bootstrap 95% CIs for TGMS—baseline: vs. B2 [+0.18, +0.59] and vs. B1 [+0.09, +0.55] (both significant); vs. B5 [0.00, +0.46] (borderline at $n=22$).

	TGMS	B1 vector-RAG	B2 static-RAG	B5 text-to-Cypher
Qwen2.5-7B, all families	0.136	0.045	0.045	0.091
Qwen2.5-14B, all families	0.409	0.091	0.045	0.182
Qwen2.5-14B, correction probes	0.67	0.00	0.00	0.00

6 Findings from live model traffic

The operator manual must be a contract. Our first live matrix run produced zero execution success across all systems-under-test rows: models emit plans that are statically valid yet reference nonexistent *output* paths (e.g., `s2.count` on an operator that emits `rows_total`). No input-schema rigor covers this. Declaring each operator’s output fields in the registry, rejecting bad paths statically, and returning the real field list in the repair payload moved probe-task execution success from 0.00 to 1.00 in one change.

Verify evidence completeness, not just values. A 14B model planned reachability without raising the page limit, computed a count over the truncated 100-row page (true answer: 343), and the claim verified as *supported*—correct arithmetic over incomplete evidence. Truncation now taints dependent steps transitively, and claims citing tainted evidence cap at weakly supported. We conjecture a family of such “evidence-quality” dimensions (cost-narrowed windows, sampling) that verification systems should track as a lattice rather than a boolean.

Serving envelopes are a fairness dimension. The standard vector-RAG configuration (top- $k=20$ chunks of 256 serialized events) assumes frontier context windows: numeric-dense event text tokenizes at ~ 0.65 tokens/character, so even $k=3$ overflows a 28k window once the decoder’s output reservation is charged. Its feasible best on this hardware ($k=1$) covers 0.4% of the corpus per question. Baseline fairness must be defined per serving envelope; the operator system’s per-task consumption ($\sim 8\text{--}10\text{k}$ tokens) is corpus-size-independent.

7 Related work

Bi-temporal agent memory (Zep/Graphiti [9]) shares the data-model precedent but treats the graph as memory retrieved into context; TGMS treats it as a database that computes, with verification against traces. **Temporal graph databases** (AeonG [3]) motivate the storage design; TGMS deliberately ships an agent-facing algebra rather than a general query language. **GraphRAG and vector RAG** are instantiated as controlled baselines here; the correction probes isolate what retrieval cannot express. **Text-to-query** systems give the model a query language directly; beyond accuracy, TGMS’s structural contrast is verifiability—claims citing raw query output cannot be machine-audited. **Temporal graph mining** algorithms [6, 11] are packaged behind verified contracts with guardrails instead of silent sampling. The tool server speaks the Model Context Protocol [1], making one deterministic, read-only temporal store shareable by many agents.

8 Limitations and roadmap

Reported numbers are development-split, single-seed, single-dataset; the frozen test campaign (more datasets, models, seeds; pre-registered thresholds recorded before any test-split run) is the immediate next step, along with an end-to-end hallucination-reduction readout against the no-verifier ablation, guided/constrained decoding for small-model plan validity, and a concurrency study of many agents sharing one store. Pattern-type claims are reported, not gated. Agent write-back with provenance is designed for (schema and log fields exist) but not enabled.

9 Conclusion

Making a temporal graph store *agent-native*—typed, deterministic, bounded operators; grounding and output contracts; trace-grounded verification; bi-temporal semantics—turns LLM answer quality from a prompting problem into a systems problem with measurable contracts. The early evidence is that the contracts do the heavy lifting: they are what small open models need to be useful, and what makes answers auditable at any model size.

Artifacts. Code, benchmark generator, task suites, the guided demo, and an auditable trace viewer: <https://github.com/zxf-work/tgms> (Apache-2.0).

References

- [1] Anthropic. Model context protocol. <https://modelcontextprotocol.io>, 2024.
- [2] Xiyang Feng, Guodong Jin, Ziyi Chen, Chang Liu, and Semih Salihoğlu. Kùzu graph database management system. In *Conference on Innovative Data Systems Research (CIDR)*, 2023.
- [3] Jiamin Hou, Zhanhao Zhang, Zhouyu Wang, Yongjun Zhang, Wei Lu, Anqun Pan, and Xiaoyong Du. Aeong: An efficient built-in temporal support in graph databases. *Proceedings of the VLDB Endowment*, 17(6):1515–1527, 2024.
- [4] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- [5] Pietro Panzarasa, Tore Opsahl, and Kathleen M. Carley. Patterns and dynamics of users’ behavior and interaction: Network analysis of an online community. *Journal of the American Society for Information Science and Technology*, 60(5):911–932, 2009.
- [6] Ashwin Paranjape, Austin R. Benson, and Jure Leskovec. Motifs in temporal networks. In *Proceedings of the Tenth ACM International Conference on Web Search and Data Mining (WSDM)*, pages 601–610, 2017.
- [7] Qwen Team. Qwen2.5 technical report, 2024. arXiv:2412.15115.
- [8] Mark Raasveldt and Hannes Mühleisen. Duckdb: An embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*, pages 1981–1984, 2019.

- [9] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory, 2025. arXiv:2501.13956.
- [10] Richard T. Snodgrass. *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann, 1999.
- [11] Huanhuan Wu, James Cheng, Silu Huang, Yiping Ke, Yi Lu, and Yanyan Xu. Path problems in temporal graphs. *Proceedings of the VLDB Endowment*, 7(9):721–732, 2014.